lockmgr API Reference

Release 1.9.4

Gene C

Sep 06, 2026

LOCKMGR API REFERENCE:

CHAPTER

ONE

LOCKMGR

1.1 Overview

lockmgr : Python Class implementing file locking

All git tags are signed with arch@sapience.com key which is available via WKD or download from https://www.sapience.com/tech. Add the key to your package builder gpg keyring. The key is included in the Arch package and the source= line with *?signed* at the end can be used to verify the git tag. You can also manually verify the signature

1.2 Recent Changes

1.9.4

- Fix missing makedepends and ensure build/install scripts fail on error Thanks to @TrialnError on Arch AUR.

1.9.1

- Use meson / meson-python for build and package management
- Periodic review
- Small code adjustments.
- Add check to PKGBUILD
- please delete any /usr/lib/python3.14/site-packages/lockmgr/__pycache__ before installing. See important note below.

** IMPORTANT **

Some older versions of this package did not include any byte compiled cache (__pycache__) and python auto creates them at runtime at a later date. If this exists, please delete the direcroty before installing this version since it now includes the .pyc files. This will avoid pacman error that a .pyc file exists. So before upgrade please do:

```
/usr/bin/rm -rf /usr/lib/python3.14/site-packages/lockmgr/__pycache__
```

My apologies.

1.3 Using LockMgr Class

Using the class is straightforward. Choose a file to use as the lockfile - avoid NFS mounts. I find using */tmp* works well since its a *tmpfs* file system.

An sample application is available in the examples directory. To see it in action, simply run *test_lock.py* in 2 different terminals. One process will acquire the lock while the other will wait until its released.

Typical usage:

```
import os
from lockmgr import LockMgr

    lockdir = '/tmp/xxx'
    os.makedirs(lockdir, exist_ok=True)
    lockfile = os.path.join(lockdir, 'foo.lock')
    lockmgr = LockMgr(lockfile)

    ...

    # Acquire lock
    if lockmgr.acquire_lock(wait=True, timeout=30):
        # do stuff
        ...

        # release lock
        lockmgr.release_lock()
   else:
        # failed to get lock
        ...
```

1.4 Research on file locking

I share my notes and research on file locking. All code samples are available in the examples/research directory.

Linux: C File Locking has linux kernel support via standard C library using fcntl(). The mechanism uses 'struct flock' as the communication mechanism with (posix) fcntl().

The standard locking mechanism uses F_SETLK. This lock lives at the process level and is the original locking in linux. Around 2015 or so 'open file desription' locks came to be via F_OFD_SETLK (See[1] and[2]). These are at the open file level. So while F_SETLK is not passed to child processes, OFD locks are. And OFD locks remain attached to the open file handle. This can be enormously useful and also surprising.

Posix locks are attached to (inode, pid) pair which means they work at the process level. If thread opens same file, even tho has different file handle, when that thread closes the fd, the lock is released for all threads in same process.

OFD locking was introduced to deal with this. Locking is attached to the "open file" and not the PID.

linux also provides "lockf" which is a wrapper around fcntl() locks - should only be used in simple cases and will interact with "normal" fcntl() locks - caveat emptor.

[1] File private locks https://lwn.net/Articles/586904/
[2] Open File Description https://lwn.net/Articles/640404/

Summary:

- Posix Lock: fcntl() - F_SETLK
- OFD Lock: fcntl() - F_OFD_SETLK
- lockf - bah

NB. The flock struct contains l_pid - this MUST be 0 for OFD locks and PID for posix locks.

Its also worth noting that locking can be file system dependent. In particular NFS should probably be avoided. Since my dominant use case is single machine, multiple process, I use /tmp which is a TMPFS file system and works well.

Python Python provides for same locking mechanisms - I recommend only 1 way for file locks in python. Python library provides for:

- fcntl.fcntl => do not use

As with C there is support for F_SETLK and F_OFD_SETLK. While these work fine, they require using 'struct' module to 'pack' and 'unpack' the C flock struct. To make this work the caller must provide the sizes (coded with letters as per the python struct module) of each element being packed.

The python 3.12 docs have examples[3], and while they may well work for a Sun workstation or similar, if you have one, the struct element sizes dont seem correct for X86_64.

I provide a little C-program to print out the correct byte sizes which you can then map to the python struct letter codes[4]

This approach is brittle - its one thing when you are coding with your own C structures, its another entirely when using system ones - these sizes should be compiled into python - while these routines work I strongly recommend not using them for this reason.

- fcntl.lockf => do not use

Wrapper around fcntl() - in spite of name this is NOT C lockf() function.

- fcntl.flock => *use this one*

Wrapper around fcntl with OFD support. i.e. this lock is associated with open file descriptor. This is what I use and recommend.

1.5 C Examples

Sample code for F_SETLK and F_OFD_SETLK To compile:

```
make
```

Builds 2 programs - *flock_sizes* and *c_lock_test*.

flock_sizes is used To print size of struct flock elements which provide the correct sizes to use in python fcntl.fcntl approach.

```
./flock_sizes
```

The test program demonstrates locking with and without OFD. To run the test progrm see the *Tests: c_lock_test* section below.

[3] Python fcntl docs: https://docs.python.org/3/library/fcntl.html
[4] Python struct module: https://docs.python.org/3/library/struct.html

Tests: c_lock_test To run locking tests, use 2 terminals. Run c_lock_test in both. The first will acquire lock while second will fail until first exits or is interrupted.

Test 1 : Using F_SETLK

```
./c_lock_test
```

Test 2 : Using F_OFD_SETLK

Repeat test but with argument to turn on OFD

```
./c_lock_test ofd
```

Test (1) and (2) both work.

1.6 Python Examples

lock_fcntl F_SETLK and F_OFD_SETLK tests in python. Run test in 2 terminals as above:

Test 3 : F_SETLK

```
./lock_fcntl.py
```

Test 4 : F_OFD_SETLK

```
./lock_fcntl.py ofd
```

Test (3) and (4) both work.

lock_flock This is what I am using. As above, run test in 2 terminals.

Test 5

```
./lock_fcntl.py
```

Test (5) works.

CHAPTER

TWO

API REFERENCE

This section contains the API guide for the `lockmgr` module.

2.1 class LockMgr

locking

`class lockmgr.LockMgr(`*lockfile*`)`

Bases: `object`

Robust file locking manager.

`acquire_lock(`*wait: bool = False*, *timeout: int = 30*`)` → bool

Try to acquire a lock.

Parameters

- `wait` – If True and timeout > 0: wait until lock is acquired (up to 10 attempts). This can be racy from the time inotify returns till we acquire lock will fail and we wll try again. If False, then do not retry if unable to acquire lock on first attempt.
- `timeout` – Number of seconds > 0 to wait between attempts to acquire the lock Will retry up to 10 times.

Returns

True if lock acquired else False

`release_lock()`

Release an Acquired Lock

No-op if there is no acquired lock.

Returns

None

CHAPTER

THREE

APPENDIX

3.1 Installation

Available on

- Github
- Archlinux AUR

On Arch you can build using the provided PKGBUILD in the packaging directory or from the AUR. To build manually, clone the repo and

```
./scripts/do-build
./scripts/do-install <destination-dir>
```

3.2 Dependencies

- Run Time :
  - python (3.14 or later)
- Building Package:
  - git
  - meson
  - meson-python
  - rsync

3.3 License

PYTHON MODULE INDEX